
<University of Kansas>

Algorithmic Algebraic Parser

User Manual

Version <1.0>

Revision History

Date	Version	Description	Author
<dd/mm/yy>	<x.x>	<details>	<name>
5/5/2026	1.0	Add the getting started section.	Simeon Geib

Table of Contents

1. Purpose	4
2. Introduction	4
3. Getting started	5
4. Advanced features	6
5. Troubleshooting	7
6. Example of uses	7
7. Glossary	9
8. FAQ	10

User Manual

1. Purpose

This document aims to provide an easy-to-understand guide on how to use the Algorithmic Algebraic Parser. The User Manual covers introductory materials on how to use the software, as well as advanced features integrated into the system. In addition, troubleshooting steps are documented with a supporting glossary and FAQ to provide comprehensive support.

2. Introduction

The Advanced Arithmetic Parser aims to be an arithmetic expression parser component in the language of C++. The objective of this software is to receive input from the user via CLI, convert the input into tokens, run correct calculations based on the tokens, and output the result. The software is also expected to gracefully handle any errors that may arise at any point in the software's process.

Features of the Advanced Arithmetic Parser include the ability to use numbers, special variables (ex. pi, e), custom functions (ex. sin, cos, tan), and operators (ex. +, -, *, /) to build and perform operations specified by the user.

Installation Process

Ensure CMake is installed. Run the command “cmake --version”. If a version number is returned, CMake is installed correctly.

Run the commands below to create and build the build directory.

- cmake -B build
- cmake --build build

The executables to run the program will be found in ./build.

CLion

CLion comes prepackaged with CMake, Ninja, and various other programs to make the compilation easy. Right-click CMakeLists in the root directory and select “Load CMake Project”. In the top left, you can select configurations and run via the build/run/debug buttons given.

Cycle Server (KU EECS)

The project can be built and run on the KU cycle server without root access using Miniconda to install CMake and the C++ compiler.

Step 1 — SSH into the cycle server

- ssh your_ku_username@cycle3.eecs.ku.edu
- Enter your KU password when prompted.

Step 2 — Install Miniconda

- wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
- bash Miniconda3-latest-Linux-x86_64.sh

During installation:

- Hold Space to scroll through the license, then type yes
- Press Enter to confirm the default install location
- When asked to initialize conda, type yes

Then reload your shell:

- `source ~/.bashrc`

Step 3 — Accept conda terms of service

- `conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main`
- `conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r`

Step 4 — Install CMake and the C++ compiler

- `conda install -c conda-forge cmake cxx-compiler -y`

Step 5 — Clone the repository

- `git clone https://github.com/348-Team-2/Term-Project.git`
- `cd Term-Project`

Step 6 — Build the project

- `mkdir build && cd build`
- `cmake ..`
- `make`

Step 7 — Run the program

- `./ParserMain`

The program is now ready to run. Enter your arithmetic expression in the command line and press Enter to view the results of the Advanced Arithmetic Parser.

3. Getting started

Usage of the Advanced Arithmetic Parser is simple:

1. Follow the process outlined in Section 2 to install the Parser.
2. Run via the "Run" button, or run the ParserMain file from the build directory.
3. Enter your expression using your keyboard. For example: `3 + 12`
4. Hit Enter or Return on your keyboard.

The Parser will then output the result of your expression. For example:

- `2 + 3` outputs 5
- `sqrt(9)` outputs 3
- `pi` outputs 3.14159...
- `2 ** 10` outputs 1024

Operators

The calculator supports the following operator types:

- Arithmetic: +, -, *, /, //, %, **
- Relational: <, <=, >, >=, ==, !=
- Logical: &&, ||, !
- Bitwise: &, |, ^, ~, <<, >>
- Implication: ->, <->
- Postfix: ! (factorial)

Functions

Functions are called using standard notation with parentheses, for example $\sin(0)$ or $\max(3, 7)$. See Section 8 FAQ for the full list of supported functions.

Constants

The following named constants can be used directly in any expression: pi, e, phi, tau, sqrt2.

Interpreting Results

If the expression is valid, the result is printed to the command line. If the expression contains an error such as division by zero or unmatched parentheses, an error message is displayed describing the problem. The program does not crash on errors. See Section 5 for troubleshooting guidance.

4. Advanced features

The Advanced Arithmetic Parser supports several features beyond standard arithmetic.

Logical and Bitwise Operators:

The calculator supports logical operators (&&, ||, !), bitwise operators (&, |, ^, ~, <<, >>), and logical implication operators (->, <->). Logical operators treat any non-zero value as true and zero as false. Bitwise operators work on the integer representations of values.

Built-in Constants:

The following constants are available for use in any expression: pi (3.14159...), e (2.71828...), phi (1.61803...), tau (6.28318...), and sqrt2 (1.41421...).

Built-in Functions:

The calculator includes 18 built-in functions: sin, cos, tan, asin, acos, atan, atan2, exp, ln, log, sqrt, abs, floor, ceil, max, min, and clamp. Functions are called using standard notation, for example $\sqrt{9}$ or $\max(3, 7)$. See the FAQ section for details on each function.

Complex Number Support:

The calculator handles complex number results automatically. For example, $\sqrt{-16}$ returns $0 + 4i$ rather than throwing an error.

5. Troubleshooting

There are two types of errors that may occur:

- Parser Error
- Evaluation Error

The Parser Error refers to an internal error in the Parser. Hopefully, you will never encounter this error type.

The far more common error type is the Evaluation Error. This means there was an error in the expression you entered into the CLI. Most of the time, our troubleshooting feature will show the location of the error in your expression. Occasionally, this will not be the case and you will be provided with a generic Evaluation Error. Either way, if you encounter this error, simply look through your expression to see what could have caused the error. Be sure to take care when entering expressions so you don't get unintended behavior.

6. Examples

<i>Name</i>	<i>Inputs</i>	<i>Outputs</i>
<i>Basic Verification</i>	$1+1$	2
<i>Right-to-Left Exponentiation</i>	$2**3**2$	512
<i>Left-to-Right Subtraction</i>	$10-4-2$	4
<i>Strict PEMDAS Enforcement</i>	$2+3*4**2$	50
<i>Extraneous Whitespace</i>	$4 * 2$	8
<i>Unary Operators</i>	$-(+1)+(+2)$	1
<i>Extraneous Parentheses</i>	$((((3+2)))$	5
<i>Sequential Unary and Binary</i>	$5+-3$	2
<i>Complex PEMDAS & Modulo</i>	$4*(3+2)\%7-1$	5
<i>Integer Evaluation Division</i>	$10/3$	3.33333
<i>Inner Parenthesis Evaluation</i>	$8-(5-2)$	5
<i>Subtraction Followed by Unary Negative</i>	$10--2$	12
<i>Addition Followed by Unary Positive</i>	$10++2$	12

<i>Testing Infinity Cutoff</i>	$1.8*10^{**307}$	$1.8e+307$
<i>Testing Infinity Cutoff</i>	$1.8*10^{**308}$	<i>inf</i>
<i>Bitwise AND</i>	$12\&10$	8
<i>Bitwise OR</i>	$12 10$	14
<i>Bitwise XOR</i>	$12\wedge 10$	6
<i>Left Bit Shift</i>	$3<<2$	12
<i>Right Bit Shift</i>	$16>>2$	4
<i>Bitwise AND with Addition</i>	$5\&3+1$	4
<i>Square Root</i>	$\text{sqrt}(16)$	4
<i>Square Root of Non-Perfect Square</i>	$\text{sqrt}(2)$	1.41421
<i>Sine</i>	$\sin(0)$	0
<i>Sine of 90 Degrees</i>	$\sin(\pi/2)$	1
<i>Cosine</i>	$\cos(0)$	1
<i>Cosine of 180 Degrees</i>	$\cos(\pi)$	-1

7. Glossary of terms

CLI — Command Line Interface. The text-based interface through which the user enters expressions and receives results.

Token — A single meaningful unit extracted from the input string, such as a number, operator, or function name.

Lexer — The pipeline stage responsible for scanning raw input text and converting it into a sequence of tokens.

Parser — The pipeline stage responsible for consuming tokens and building an Abstract Syntax Tree that encodes the mathematical structure and precedence of the expression.

AST (Abstract Syntax Tree) — A hierarchical tree structure produced by the Parser that represents the grammatical structure of the input expression.

Evaluator — The pipeline stage responsible for recursively traversing the AST and computing the final result by delegating mathematical execution to the Registry.

Registry — The central repository that stores all operator definitions, function logic, and constant values. The Lexer, Parser, and Evaluator all query the Registry.

PEMDAS — Parentheses, Exponents, Multiplication, Division, Addition, Subtraction. The standard order of mathematical operations enforced by the Parser.

ParseError — An exception thrown when the input expression violates the grammar rules of the parser, for example unmatched parentheses or a missing operand.

EvalError — An exception thrown when a mathematical error occurs during evaluation, for example division by zero or a domain error in a function.

8. FAQ

1. What is the pipeline and how does input become a result?

Input flows through five sequential stages: Lexer → Tokens → Parser → AST → Evaluator → Output. The Lexer converts raw text into tokens, the Parser builds a tree structure encoding precedence, and the Evaluator walks that tree to produce a final result.

2. Why does the Evaluator delegate math to the Registry instead of doing it itself?

To keep the system extensible without modifying core code. All operator and function logic lives in the Registry, so new math operations can be added to math.cpp without touching the Evaluator. The Evaluator only determines what kind of node it is looking at — the Registry determines what the math does.

3. What is EvalError and when is it thrown?

EvalError is thrown when the Evaluator encounters an unrecoverable state during execution. This happens in two cases: when the AST contains an unrecognized node type, or when the Registry reports an invalid math state such as division by zero, a domain error, or an incorrect number of function arguments. The program does not crash — EvalError is caught by the CLI and displayed as a clean error message.

4. What kinds of expressions does the calculator support?

The calculator supports arithmetic, bitwise, logical, relational, and implication operators, as well as prefix and postfix unary operators, multi-argument functions, and named constants. It does not support variable assignment or multi-line expressions. All input must be entered as a single expression on the command line.

5. Does the calculator follow standard order of operations?

Yes. The calculator enforces full PEMDAS precedence. Exponentiation binds tighter than multiplication and division, which bind tighter than addition and subtraction. Parentheses can override any precedence. $2 + 3 * 4$ evaluates to 14, not 20.

6. What is the difference between / and //?

/ performs standard floating point division and returns a decimal result. // performs floor division and truncates toward zero, returning a whole number. $10 / 3$ returns 3.333... while $10 // 3$ returns 3.

7. What is the difference between ^ and **?

** is the exponentiation operator. ^ is the bitwise XOR operator. $2 ** 3$ returns 8. $2 ^ 3$ returns 1 because it operates on the binary representations of the integers. If you intend to raise a number to a power, always use **.

8. What does 5! evaluate to, and how is it different from !5?

5! is the postfix factorial operator and evaluates to 120. !5 is the prefix logical NOT operator — since 5 is non-zero it is treated as true, so !5 evaluates to 0. They use the same symbol but are distinguished entirely by position.

9. What is the difference between & and &&?

& is bitwise AND and operates on the binary representations of integers. && is logical AND and treats both operands as booleans, returning 1 if both are non-zero and 0 otherwise. $6 \& 3$ returns 2. $6 \&\& 3$ returns 1.

10. What happens when you divide by zero?

The program does not crash. Division by zero is caught before execution and thrown as an `EvalError` with the message "Division by zero." The CLI displays the error message and waits for the next input. This applies to both `/` and `//`.

11. What does `sqrt(-16)` return?

The `sqrt` function detects a negative input and returns a complex number instead of throwing an error. `sqrt(-16)` returns `0 + 4i` as a `std::complex<double>`. This is supported by the calculator's `Value` type, which handles complex numbers alongside standard scalars.

12. What does `---2` evaluate to and why?

`---2` evaluates to `-2`. Each `-` is treated as a prefix unary negation operator. The parser allows stacked prefix operators, so `---2` is parsed as `-(-(- 2))`. The innermost `-2` gives `-2`, negated gives `2`, negated again gives `-2`. An even number of negations cancels out, an odd number does not.

13. What happens if you call a function with the wrong number of arguments?

The program does not crash. The Registry strictly validates argument count before executing any function. If the count does not match, an `std::invalid_argument` is thrown with a message identifying the function and how many arguments were expected. The Evaluator catches this and re-throws it as an `EvalError`, which the CLI displays as a clean error message. For example, calling `max(1, 2, 3)` would produce: Evaluation Error: Function 'max' expected 2 arguments.

14. What does `->` mean?

`->` is the logical implication operator. It returns `0` (false) only when the left side is true and the right side is false — in all other cases it returns `1` (true). This follows standard propositional logic: `p -> q` means "if `p` then `q`." For example `1 -> 0` returns `0`, but `0 -> 0`, `0 -> 1`, and `1 -> 1` all return `1`. The related operator `<->` is biconditional — it returns `1` only when both sides have the same boolean value.